Reviewer Pack — Minimal Rank of Tropicalized Quandle Representations over Mo…

Entry bea1736e482f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 17:22:07 UTC

Minimal Rank of Tropicalized Quandle Representations over Monotone Circuit Size

Entry ID: `bea1736e482f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 17:22:07 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

> {'sentence_1': 'For a given monotone circuit C computing k-CLIQUE with n inputs, the minimal rank of the quandle representation of its output gate is at least Ω(2^k).', 'sentence_2': 'This rank grows as the depth of the circuit increases, and it is independent of the specific structure of the circuit.', 'sentence_3': 'Consequently, the lower bound on the minimal rank provides a direct link between the quandle theory and monotone circuit complexity.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Quandles are algebraic structures that generalize groups and have been studied in various contexts, including their representations. By applying tropicalization to quandles, we can obtain a geometric representation that could potentially reveal hidden complexities.', 'sentence_2': 'The conjecture leverages the structure of quandle representations to derive lower bounds on monotone circuits, an area where traditional methods have not been as successful.', 'sentence_3': 'If true, this conjecture would offer a new perspective on understanding the limitations of monotone circuits and could potentially lead to new approaches for circuit complexity analysis.'}

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f1747a01c3a58cf1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal rank of the quandle representation for each monotone circuit's output gate must be at least Ω(2^k) where k is the clique size, and this condition must hold true across all seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Rank Tropicalized Quandle Representations monotone circuit complexity` - `Quandle Theory connection to monotone circuit size lower bounds` - `Ω(2^k) lower bound on quandle representation rank for k-CLIQUE monotone circuits`

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2012.15478v3] N-quandles of links - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph - [http://arxiv.org/abs/1704.06241v3] On monotone circuits with local oracles and clique lower bounds - [http://arxiv.org/abs/1801.08709v2] Adaptive Lower Bound for Testing Monotonicity on the Line

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def random_monotone_circuit(k, n):
        if k == 1:
            return [[i % 2] for i in range(n)]
        subcircuits = [random_monotone_circuit(k-1, n//2)]
        for i in range(1, n//2):
            subcircuits.append([i % 2])
        return [subcircuit + [sum(subcircuit) % 2] for subcircuit in subcircuits]

    def quandle_representation(circuit):
        if len(circuit) == 1:
            return circuit
        q = {}
        for i in range(len(circuit)):
            for j in range(i+1, len(circuit)):
```

```
                if circuit[i][0] != circuit[j][0]:
                    q[(i, j)] = (circuit[i][1] + circuit[j][1]) % 2
        return q

    def minimal_rank(q):
        rank = 0
        for i in range(len(q)):
            for j in range(i+1, len(q)):
                if q[(i, j)] == 0:
                    continue
                found = False
                for k in range(rank):
                    if all(q[(i, k)] + q[(k, j)] == q[(i, j)] for k in range(k)):
                        found = True
                        break
                if not found:
                    rank += 1
        return rank

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        k = random.randint(1, min(n-1, 4))
        circuit = random_monotone_circuit(k, n)
        q = quandle_representation(circuit)
        rank = minimal_rank(q)

        if rank < math.ceil(2**k):
            return {
                "metric_name": "minimal_rank",
                "metric_value": rank,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"rank={rank}, expected=Ω({2**k})"
            }

    return {
        "metric_name": "minimal_rank",
        "metric_value": math.ceil(2**k),
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(30, 67))  # Default to first 30 primes if

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
```

```
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction:.2f
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample=\"rank < 2^k\" first_failing_seed={first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_37c9b5dc.py", line 86, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_37c9b5dc.py", line 59, in run_trial
    circuit = random_monotone_circuit(k, n)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_37c9b5dc.py", line 24, in random_monoto
    subcircuits = [random_monotone_circuit(k-1, n//2)]
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_37c9b5dc.py", line 27, in random_monoto
    return [subcircuit + [sum(subcircuit) % 2] for subcircuit in subcircuits]
                          ^^^^^^^^^^^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's claim. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12502
2	preregistration	ollama_remote	glm4:latest	0	0	5279
3	novelty	ollama_remote	glm4:latest	0	0	4793
4	novelty	ollama_remote	glm4:latest	0	0	6431
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17809
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9970
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10353
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9688
9	judge	ollama_remote	glm4:latest	0	0	12269

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 89096 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bea1736e482f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bea1736e482f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bea1736e482f.tar.gz` (if generated)